

SCR2043 OPERATING SYSTEMS

Name : NABIL AFLAH BOO BINTI
MOHD YOSUF BOO YONG
CHONG
Student ID : A23CS0252
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
<code>ps -e</code>
Output

```
aflah@secr2043:~$ ps -e
  PID TTY          TIME CMD
    1 ?           00:00:15 systemd
    2 ?           00:00:00 kthreadd
    3 ?           00:00:00 pool_workqueue_release
    4 ?           00:00:00 kworker/R-rcu_g
    5 ?           00:00:00 kworker/R-rcu_p
    6 ?           00:00:00 kworker/R-slub_
    7 ?           00:00:00 kworker/R-netns
   11 ?           00:00:00 kworker/u4:0-floppy
   12 ?           00:00:00 kworker/R-mm_pe
   13 ?           00:00:00 rcu_tasks_kthread
   14 ?           00:00:00 rcu_tasks_rude_kthread
   15 ?           00:00:00 rcu_tasks_trace_kthread
   16 ?           00:01:28 ksoftirqd/0
   17 ?           00:00:02 rcu_preempt
   18 ?           00:00:00 migration/0
   19 ?           00:00:00 idle_inject/0
   20 ?           00:00:00 cpuhp/0
   21 ?           00:00:00 cpuhp/1
   22 ?           00:00:00 idle_inject/1
   23 ?           00:00:00 migration/1
   24 ?           00:00:00 ksoftirqd/1
   26 ?           00:00:00 kworker/1:0H-events_highpri
   27 ?           00:00:00 kworker/u5:0-events_unbound
   29 ?           00:00:00 kdevtmpfs
   30 ?           00:00:00 kworker/R-inet_
   32 ?           00:00:00 kauditd
   33 ?           00:00:00 khungtaskd
   34 ?           00:00:00 oom_reaper
   35 ?           00:00:01 kworker/u5:2-events_power_efficient
   36 ?           00:00:00 kworker/R-write
   37 ?           00:00:00 kcompactd0
   38 ?           00:00:00 ksmd
   40 ?           00:00:00 khugepaged
   41 ?           00:00:00 kworker/R-kinte
   42 ?           00:00:00 kworker/R-kbloc
   43 ?           00:00:00 kworker/R-blkcg
   44 ?           00:00:00 irq/9-acpi
   45 ?           00:00:00 kworker/R-tpm_d
   46 ?           00:00:00 kworker/R-ata_s
   47 ?           00:00:00 kworker/R-md
```

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
<code>ps -ef</code>
Output

```

aflah@secr2043:~$ ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root           1         0  0  14:14 ?        00:00:15 /sbin/init
root           2         0  0  14:14 ?        00:00:00 [kthreadd]
root           3          2  0  14:14 ?        00:00:00 [pool_workqueue_rele
root           4          2  0  14:14 ?        00:00:00 [kworker/R-rcu_g]
root           5          2  0  14:14 ?        00:00:00 [kworker/R-rcu_p]
root           6          2  0  14:14 ?        00:00:00 [kworker/R-slab_]
root           7          2  0  14:14 ?        00:00:00 [kworker/R-netns]
root          11          2  0  14:14 ?        00:00:00 [kworker/u4:0-floppy
root          12          2  0  14:14 ?        00:00:00 [kworker/R-mm_pe]
root          13          2  0  14:14 ?        00:00:00 [rcu_tasks_kthread]
root          14          2  0  14:14 ?        00:00:00 [rcu_tasks_rude_kthr
root          15          2  0  14:14 ?        00:00:00 [rcu_tasks_trace_kth
root          16          2  3  14:14 ?        00:01:28 [ksoftirqd/0]
root          17          2  0  14:14 ?        00:00:02 [rcu_preempt]
root          18          2  0  14:14 ?        00:00:00 [migration/0]
root          19          2  0  14:14 ?        00:00:00 [idle_inject/0]
root          20          2  0  14:14 ?        00:00:00 [cpuhp/0]
root          21          2  0  14:14 ?        00:00:00 [cpuhp/1]
root          22          2  0  14:14 ?        00:00:00 [idle_inject/1]
root          23          2  0  14:14 ?        00:00:00 [migration/1]
root          24          2  0  14:14 ?        00:00:00 [ksoftirqd/1]
root          26          2  0  14:14 ?        00:00:00 [kworker/1:0H-events
root          27          2  0  14:14 ?        00:00:00 [kworker/u5:0-events
root          29          2  0  14:14 ?        00:00:00 [kdevtmpfs]
root          30          2  0  14:14 ?        00:00:00 [kworker/R-inet_]
root          32          2  0  14:14 ?        00:00:00 [kauditd]
root          33          2  0  14:14 ?        00:00:00 [khungtaskd]
root          34          2  0  14:14 ?        00:00:00 [oom_reaper]
root          35          2  0  14:14 ?        00:00:01 [kworker/u5:2-events
root          36          2  0  14:14 ?        00:00:00 [kworker/R-write]
root          37          2  0  14:14 ?        00:00:00 [kcompactd0]
root          38          2  0  14:14 ?        00:00:00 [ksmd]
root          40          2  0  14:14 ?        00:00:00 [khugepaged]
root          41          2  0  14:14 ?        00:00:00 [kworker/R-kinte]
root          42          2  0  14:14 ?        00:00:00 [kworker/R-kbloc]
root          43          2  0  14:14 ?        00:00:00 [kworker/R-blkcg]
root          44          2  0  14:14 ?        00:00:00 [irq/9-acpi]
root          45          2  0  14:14 ?        00:00:00 [kworker/R-tpm_d]

```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
<code>ps aux grep subprocess grep -v grep</code>
Output

```
aflah@secr2043:~$ ps aux | grep subprocess | grep -v grep
aflah      1245  0.0  0.1   2680  1536 ?        S      14:43   0:00 ./subprocess1
aflah      1246  0.0  0.1   2680  1536 ?        S      14:43   0:00 ./subprocess1
aflah      1248  0.0  0.1   2680  1536 ?        S      14:43   0:00 ./subprocess2
aflah      1249  0.0  0.1   2680  1408 ?        S      14:43   0:00 ./subprocess2
aflah      1250  0.0  0.1   2680  1536 ?        S      14:43   0:00 ./subprocess2
aflah      1253  0.0  0.1   2680  1536 ?        S      14:43   0:00 ./subprocess1
aflah      1254  0.0  0.1   2680  1536 ?        S      14:44   0:00 ./subprocess2
aflah      1284  0.0  0.1   2680  1536 pts/1    S      14:52   0:00 ./subprocess1
aflah      1285  0.0  0.1   2680  1536 pts/1    S      14:52   0:00 ./subprocess1
aflah      1286  0.0  0.1   2680  1536 pts/1    S      14:52   0:00 ./subprocess2
aflah      1287  0.0  0.1   2680  1536 pts/1    S      14:52   0:00 ./subprocess2
aflah      1288  0.0  0.1   2680  1536 pts/1    S      14:52   0:00 ./subprocess2
aflah      1289  0.0  0.1   2680  1536 pts/1    S      14:52   0:00 ./subprocess1
aflah      1290  0.0  0.1   2680  1536 pts/1    S      14:53   0:00 ./subprocess2
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
<code>ps -eo pid,user,pcpu,pmem,cmd</code>
Output

```

aflah@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd
  PID  USER      %CPU  %MEM  CMD
    1   root        0.6   1.3  /sbin/init
    2   root        0.0   0.0  [kthreadd]
    3   root        0.0   0.0  [pool_workqueue_release]
    4   root        0.0   0.0  [kworker/R-rcu_g]
    5   root        0.0   0.0  [kworker/R-rcu_p]
    6   root        0.0   0.0  [kworker/R-slub_]
    7   root        0.0   0.0  [kworker/R-netns]
   11   root        0.0   0.0  [kworker/u4:0-floppy]
   12   root        0.0   0.0  [kworker/R-mm_pe]
   13   root        0.0   0.0  [rcu_tasks_kthread]
   14   root        0.0   0.0  [rcu_tasks_rude_kthread]
   15   root        0.0   0.0  [rcu_tasks_trace_kthread]
   16   root        3.6   0.0  [ksoftirqd/0]
   17   root        0.0   0.0  [rcu_preempt]
   18   root        0.0   0.0  [migration/0]
   19   root        0.0   0.0  [idle_inject/0]
   20   root        0.0   0.0  [cpuhp/0]
   21   root        0.0   0.0  [cpuhp/1]
   22   root        0.0   0.0  [idle_inject/1]
   23   root        0.0   0.0  [migration/1]
   24   root        0.0   0.0  [ksoftirqd/1]
   26   root        0.0   0.0  [kworker/1:0H-events_highpri]
   27   root        0.0   0.0  [kworker/u5:0-flush-8:0]
   29   root        0.0   0.0  [kdevtmpfs]
   30   root        0.0   0.0  [kworker/R-inet_]
   32   root        0.0   0.0  [kauditd]
   33   root        0.0   0.0  [khungtaskd]
   34   root        0.0   0.0  [oom_reaper]
   35   root        0.0   0.0  [kworker/u5:2-events_power_efficient]
   36   root        0.0   0.0  [kworker/R-write]
   37   root        0.0   0.0  [kcompactd0]
   38   root        0.0   0.0  [ksmd]
   40   root        0.0   0.0  [khugepaged]
   41   root        0.0   0.0  [kworker/R-kinte]
   42   root        0.0   0.0  [kworker/R-kbloc]
   43   root        0.0   0.0  [kworker/R-blkcq]
   44   root        0.0   0.0  [irq/9-acpi]
   45   root        0.0   0.0  [kworker/R-tpm_d]
   46   root        0.0   0.0  [kworker/R-ata_s]

```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (pmem)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem</code>
Output

```

aflah@secre2043:~$ ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem
PID USER      %CPU %MEM    CMD
1303 root        1.7  4.1  /usr/libexec/fwupd/fwupd
352  root        0.4  2.7  /sbin/multipathd -d -s
789  root        0.0  2.3  /usr/bin/python3 /usr/share/unattended-upgrad
307  root        0.1  1.7  /usr/lib/systemd/systemd-journald
674  root        0.0  1.3  /usr/libexec/udisks2/udisksd
1    root        0.6  1.3  /sbin/init
545  systemd+   0.0  1.2  /usr/lib/systemd/systemd-resolved
776  root        0.0  1.2  /usr/sbin/ModemManager
1035 student    0.0  1.1  /usr/lib/systemd/systemd --user
653 polkitd    0.0  0.9  /usr/lib/polkit-1/polkitd --no-debug
528  systemd+   0.0  0.9  /usr/lib/systemd/systemd-networkd
665  root        0.0  0.8  /usr/lib/systemd/systemd-logind
1265 root        0.0  0.8  sshd: aflah [priv]
1113 root        0.0  0.8  sshd: /usr/sbin/sshd -D [listener] 0 of 10-10
548  systemd+   0.0  0.7  /usr/lib/systemd/systemd-timesyncd
372  root        0.0  0.7  /usr/lib/systemd/systemd-udev
1267 aflah      0.8  0.7  sshd: aflah@pts/1
722  syslog     0.0  0.5  /usr/sbin/rsyslogd -n -iNONE
647  message+   0.0  0.5  @dbus-daemon --system --address=systemd: --no
1084 student    0.0  0.5  -bash
1268 aflah      0.0  0.5  -bash
830  root        0.0  0.4  /bin/login -p --
1343 aflah     100  0.4  ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem
1036 student    0.0  0.3  (sd-pam)
646  root        0.0  0.2  /usr/sbin/cron -f -P
1243 aflah      0.0  0.1  ./mainprocess
1245 aflah      0.0  0.1  ./subprocess1
1246 aflah      0.0  0.1  ./subprocess1
1248 aflah      0.0  0.1  ./subprocess2
1250 aflah      0.0  0.1  ./subprocess2
1253 aflah      0.0  0.1  ./subprocess1
1254 aflah      0.0  0.1  ./subprocess2
1281 aflah      0.0  0.1  ./mainprocess
1284 aflah      0.0  0.1  ./subprocess1
1285 aflah      0.0  0.1  ./subprocess1
1286 aflah      0.0  0.1  ./subprocess2
1287 aflah      0.0  0.1  ./subprocess2
1288 aflah      0.0  0.1  ./subprocess2
1289 aflah      0.0  0.1  ./subprocess1
1290 aflah      0.0  0.1  ./subprocess2
1249 aflah      0.0  0.1  ./subprocess2
1244 aflah      0.0  0.0  ./mainprocess
1247 aflah      0.0  0.0  ./mainprocess
1282 aflah      0.0  0.0  ./mainprocess
1283 aflah      0.0  0.0  ./mainprocess

```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
<code>ps -ejH grep -E "mainprocess subprocess1 subprocess2"</code>
Output

```
aflah@secr2043:~$ ps -ejH | grep -E "mainprocess|subprocess1|subprocess2"
1281 1281 1268 pts/1 00:00:00 mainprocess
1282 1281 1268 pts/1 00:00:00 mainprocess
1284 1281 1268 pts/1 00:00:00 subprocess1
1285 1281 1268 pts/1 00:00:00 subprocess1
1283 1281 1268 pts/1 00:00:00 mainprocess
1286 1281 1268 pts/1 00:00:00 subprocess2
1287 1281 1268 pts/1 00:00:00 subprocess2
1288 1281 1268 pts/1 00:00:00 subprocess2
1289 1289 1268 pts/1 00:00:00 subprocess1
1290 1290 1268 pts/1 00:00:00 subprocess2
1243 1243 1216 ? 00:00:00 mainprocess
1244 1243 1216 ? 00:00:00 mainprocess
1245 1243 1216 ? 00:00:00 subprocess1
1246 1243 1216 ? 00:00:00 subprocess1
1247 1243 1216 ? 00:00:00 mainprocess
1248 1243 1216 ? 00:00:00 subprocess2
1249 1243 1216 ? 00:00:00 subprocess2
1250 1243 1216 ? 00:00:00 subprocess2
1253 1253 1216 ? 00:00:00 subprocess1
1254 1254 1216 ? 00:00:00 subprocess2
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
<code>pstree -p -t</code>
Output

Command
<code>pskill mainprocess</code>
Output
<pre> aflah@secr2043:~\$ pskill mainprocess Main process (ID: 1281) received signal: 15. Terminating... Main process (ID: 1282) received signal: 15. Terminating... Main process (ID: 1283) received signal: 15. Terminating... [1] Done ./mainprocess </pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if(pid == 0){
        printf("Child process: PID = %d\n", getpid());
    } else {
        wait(NULL);
        printf("Parent process: PID = %d\n", getpid());
    }
    return 0;
}

```

Output:

```
aflah@secr2043:~$ gcc forkwait.c -o forkwait
aflah@secr2043:~$ ./forkwait
Child process: PID = 1514
Parent process: PID = 1513
```